

Natural Language Processing

Hidden Markov Models and Word2Vec

Why Study These Two Together?

Classical	Neural
Probabilistic	Vector space
Interpretable	Dense
Sequence modeling	Representation learning

NLP Problems That Need Sequences

Examples:

- POS Tagging
- Named Entity Recognition
- Speech recognition
- Machine translation

Hidden Markov Models

- A probabilistic model where:
 - Hidden state generate observations
 - States follow Markov property

- Components:

- States -> POS Tags
- Observations -> Words
- Parameters:
 - Transition matrix A
 - Emission matrix B
 - Initial distribution π

- We want: $\arg \max_{tags} P(tags|words)$

- Solution: Viterbi Algorithm

$$P(q_t|q_{t-1}, q_{t-2}) = P(q_t|q_{t-1})$$

POS Tagging as HMM:

Sentence: *The dog runs*

Hidden states: *DET NOUN VERB*

Implementation Details

Imports

```
import nltk
import numpy as np
from collections import defaultdict
from nltk.corpus import brown
```

Load tagged sentences

```
sentences = brown.tagged_sents(tagset='universal')
```

Build counts

```
transition_counts = defaultdict(lambda: defaultdict(int))
emission_counts = defaultdict(lambda:
defaultdict(int))tag_counts = defaultdict(int)
```

```
for sent in sentences:
    prev_tag = "<START>"
    for word, tag in sent:
        transition_counts[prev_tag][tag] += 1
        emission_counts[tag][word.lower()] += 1
        tag_counts[tag] += 1
        prev_tag = tag
    transition_counts[prev_tag]["<END>"] += 1
```

I

```
# Convert to probabilities
```

```
transition_probs = defaultdict(dict)
emission_probs = defaultdict(dict)
```

```
for prev_tag in transition_counts:
    total = sum(transition_counts[prev_tag].values())
    for tag in transition_counts[prev_tag]:
        transition_probs[prev_tag][tag] = transition_counts[prev_tag][tag] / total
```

```
for tag in emission_counts:
    total = tag_counts[tag]
    for word in emission_counts[tag]:
        emission_probs[tag][word] = emission_counts[tag][word] / total
```

```

def viterbi(sentence, tags, transition_probs, emission_probs):
    V = [{}]
    path = {}

    for tag in tags:
        V[0][tag] = transition_probs["<START>"].get(tag, 1e-6) * \
            emission_probs[tag].get(sentence[0], 1e-6)
        path[tag] = [tag]

    for t in range(1, len(sentence)):
        V.append({})
        new_path = {}

        for curr_tag in tags:
            (prob, state) = max(
                (V[t-1][prev_tag] *
                 transition_probs[prev_tag].get(curr_tag, 1e-6) *
                 emission_probs[curr_tag].get(sentence[t], 1e-6),
                 prev_tag)
                for prev_tag in tags
            )

            V[t][curr_tag] = prob
            new_path[curr_tag] = path[state] + [curr_tag]

        path = new_path

    (prob, state) = max((V[len(sentence)-1][tag], tag) for tag in tags)
    return path[state]

```

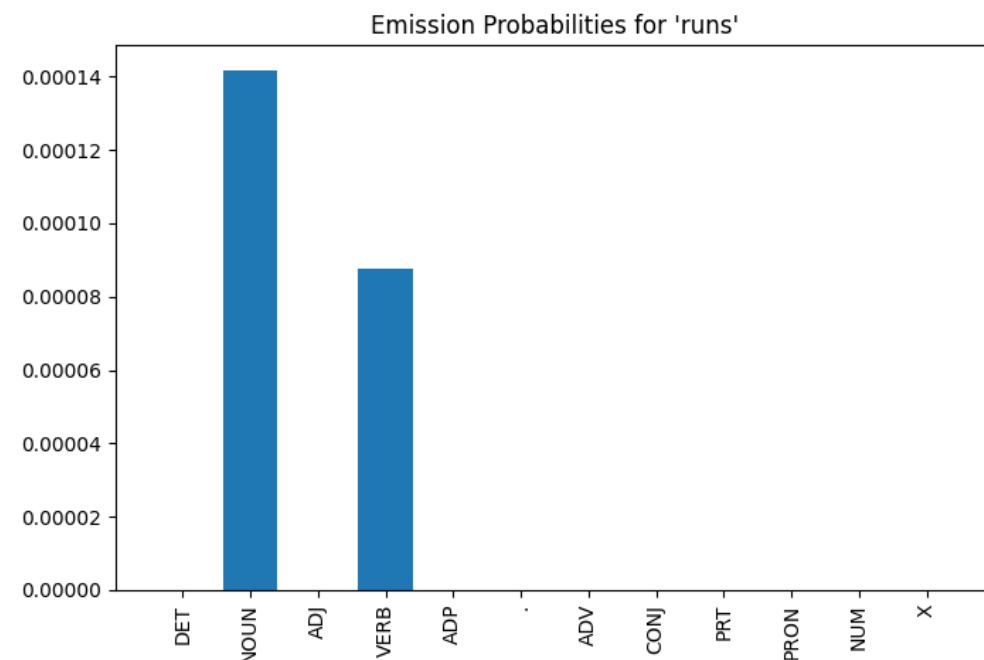
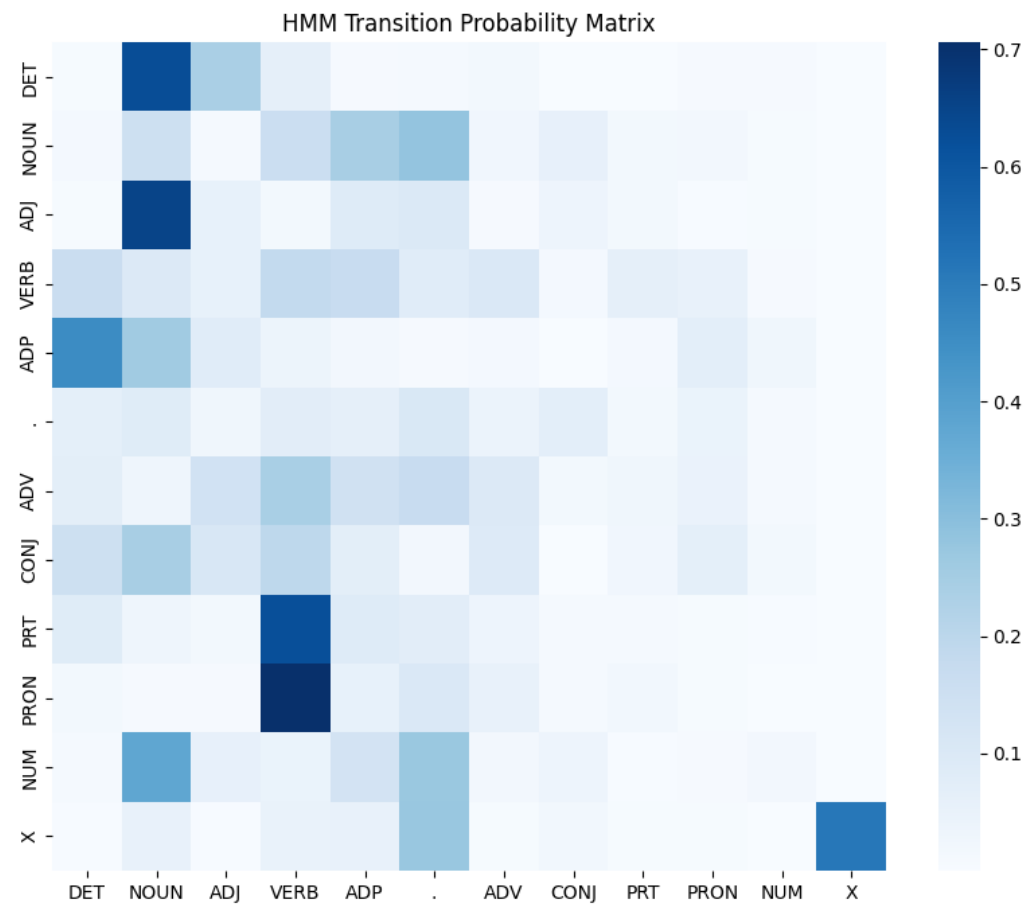
Viterbi Algorithm

```

tags = list(tag_counts.keys())
sentence = "the dog runs".split()
print(viterbi(sentence, tags, transition_probs, emission_probs))

```

Testing



Word2Vec: Word Embeddings

- **Why Not One-Hot?**
- Example: King = [0,0,0,0,1,0...]
- Problems:
 - Sparse
 - No similarity

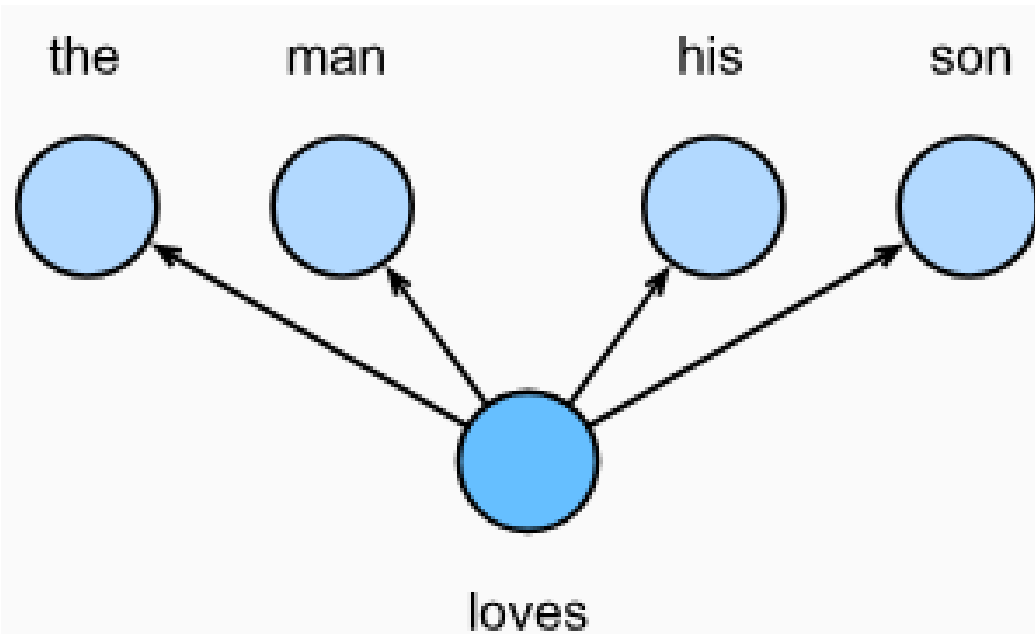
Core Idea: Word2Vec

Predict:

1. Context from word (Skip-Gram)
2. Word from context (CBOW)

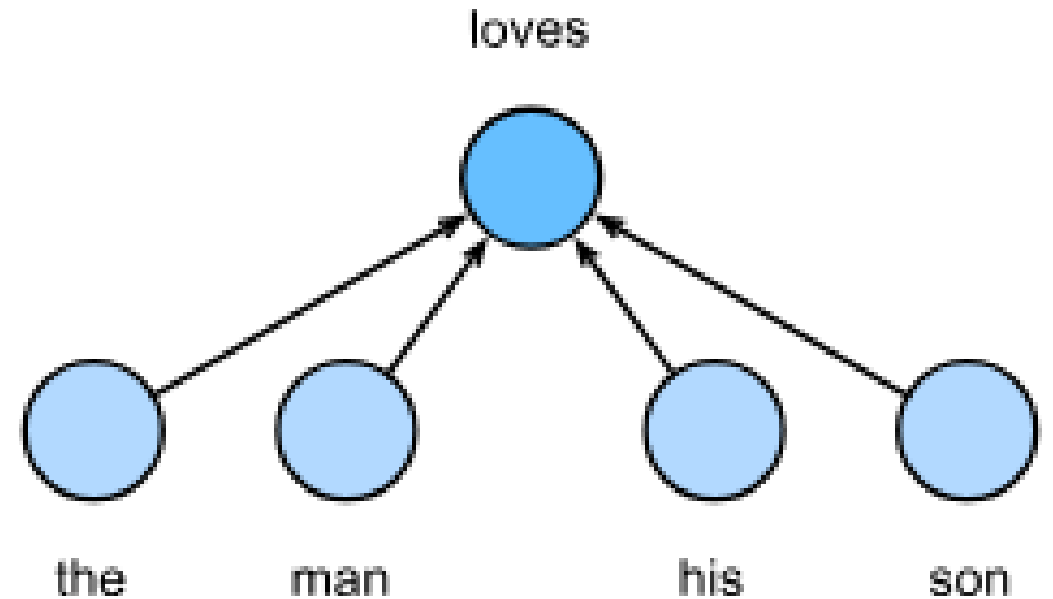
Skip-Gram

$P(\text{"the", "man", "his", "son"} \mid \text{"loves"})$.



Continuous Bag of Words

$P(\text{"loves"} \mid \text{"the", "man", "his", "son"})$.



Implementation Details

Imports

```
from gensim.models import Word2Vec
from nltk.corpus import brown
```

Training

```
# Prepare sentences
sentences = [[word.lower() for word in sent] for sent in brown.sents()]

# Train model
model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=5,
    workers=4
)
```

I

```
# Train model
model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    epochs=20
)
```

Continuous Bag of Words

Skip-gram

```
model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    sg=1,           # Skip-gram
    epochs=20
)
```

Testing

Similarity

Similar words

I

```
print(model.wv.most_similar("king"))
```

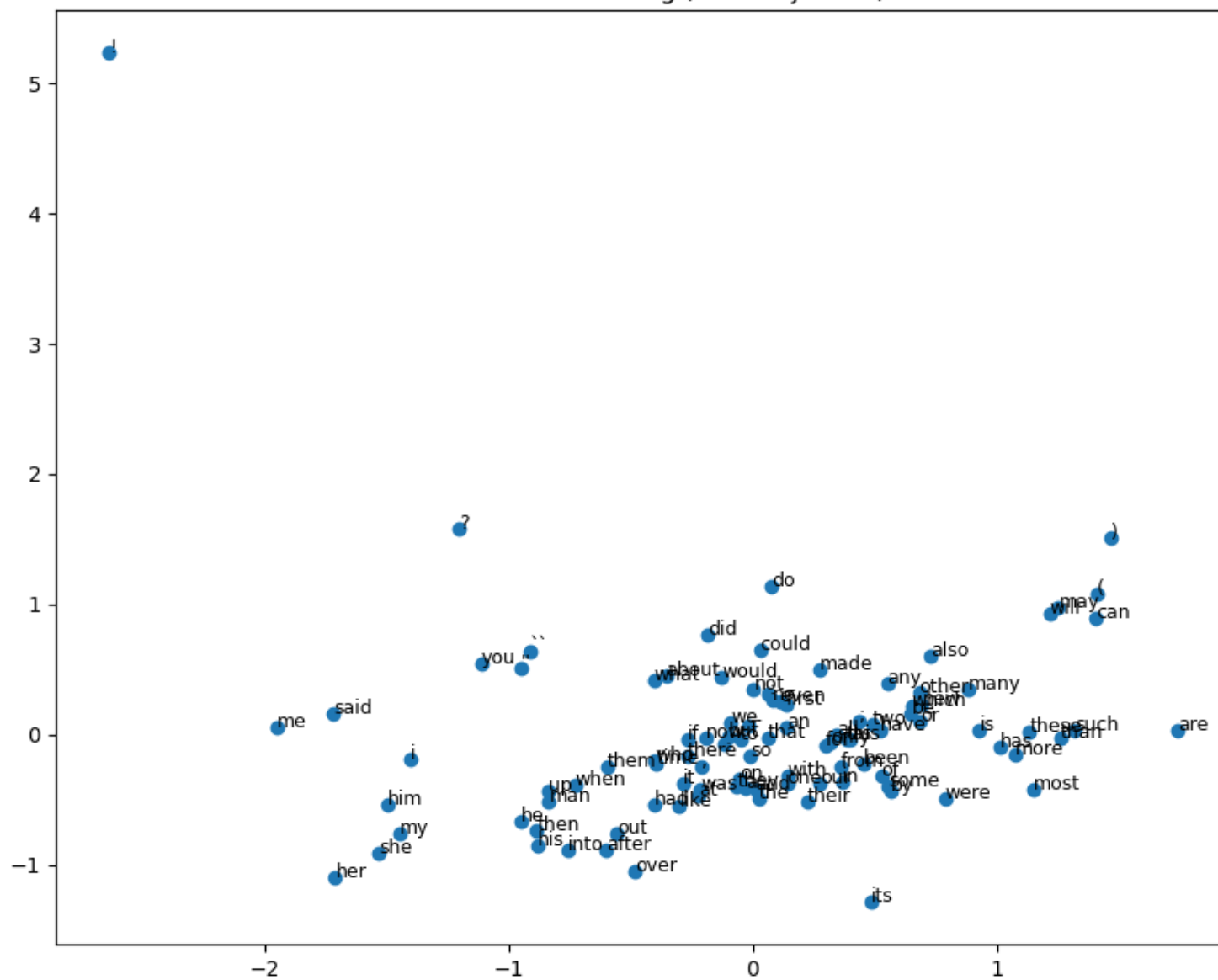
```
[('richard', 0.7332358360290527), ('henry', 0.7011101841926575), ('quincy', 0.6959963440895081), ('mary', 0.6896296739578247), ('william', 0.6806308031082153), ('judge', 0.6700136661529541), ('thompson', 0.66913241147995), ('peter', 0.6683146357536316), ('ambassador', 0.663901150226593), ('senator', 0.663835883140564)]
```

Analogy

```
print(model.wv.most_similar(  
    positive=["king", "woman"],  
    negative=["man"]  
))
```

```
[('mary', 0.6637348532676697), ('maid', 0.6320381164550781), ('thompson', 0.6299006938934326), ('uncle', 0.6147308945655823), ('virgin', 0.6137285828590393), ('judy', 0.6068179607391357), ('richard', 0.604846715927124), ('steichen', 0.5993967056274414), ('giles', 0.5988671779632568), ('jane', 0.5980986952781677)]
```

Word2Vec Embedding (PCA Projection)



Hidden Markov Models vs Word2Vec

HMM	Word2Vec
Generative	Predictive
Sequence model	Embedding model
Probabilistic	Neural
Requires labels	Self-supervised
Good for tagging	Better Semantics
Interpretable	Captures analogies

Assignments

1. Improve HMM
 1. Add Laplace smoothing in transition and emission probabilities $((\text{count} + 1) / \text{total})$
 2. Handle Unknown words
 3. Compute accuracy on 100 test sentences
2. Train and compare 2 Word2Vec models
 1. 50 dimensional
 2. 200 dimensional
3. (Optional)
 1. Word clustering using K-Means for Word2Vec
 2. Sentence similarity for Word2Vecs